PYTHON FOR DATA SCIENCE

A beginner's course companion and practice handbook

Based on: freeCodeCamp.org, *Python for Data Science - Course for Beginners (Learn Python, Pandas, NumPy, Matplotlib)*
Instructor: Maxwell Armi | **Video length:** 12h 19m 52s | **Level:** Beginner
Original video: https://www.youtube.com/watch?v=LHBE6Q9XlzI
Official course code: https://github.com/datapublishings/Course-python-data-science

> **How these notes were prepared:** The chapter sequence and start times come from the video's published description. The topics and project were cross-checked against the course's public Jupyter notebooks. These are original teaching notes and updated, runnable examples - not a verbatim transcript or a claim to have viewed every minute of the recording.

How to use this guide

1. **Read** one section, then watch the matching chapter at the timestamp shown.
2. **Type** every example yourself in a Python file or Jupyter notebook; do not just read it.
3. **Change** a number, condition, or column name and predict what the output will be.
4. **Finish** the mini-practice tasks. Use the answer key only after trying.

Learning path: Problem solving -> Python foundations -> Data structures -> NumPy -> Pandas -> Matplotlib -> A complete data analysis workflow.

What this course does not focus on: advanced object-oriented programming, web development, and comprehensive exception handling. You can study those separately after these foundations.

Video chapter map

Start	Chapter	Main goal
00:00:00	Introduction	Understand course roadmap
00:03:53	Basics of programming	Algorithms, pseudocode, problem solving
01:11:35	Why Python	Why Python suits data analysis
01:33:09	Install Anaconda and Python	Set up your environment
01:37:25	Launch Jupyter Notebook	Write and run notebook cells
01:46:28	iPython Shell	Run interactive Python commands

Start	Chapter	Main goal
01:53:33	Variables and operators	Store data and perform calculations
02:27:45	Booleans and comparisons	Build true/false conditions
02:55:37	Useful Python functions	Speed up common operations
03:20:04	Control flow	if/else and loops
05:11:52	Functions	Write reusable logic
06:41:47	Modules	Import existing code
07:30:04	Strings	Work with text
08:23:57	Lists, tuples, sets, dictionaries	Choose appropriate containers
09:36:10	NumPy	Fast numerical arrays
11:04:12	Pandas	Explore and transform tabular data
12:01:31	Matplotlib	Create charts
12:09:00	Project	Analyze COVID-19 trends

PART I - THINKING LIKE A PROGRAMMER

1. Course introduction (00:00:00)

Python is a general-purpose programming language. **Data science** uses data and statistical reasoning to answer questions, find patterns, and make informed decisions. In this course, Python provides the language, while **NumPy**, **Pandas**, and **Matplotlib** provide powerful data-science tools.

A useful mental model:

- **Python fundamentals:** tell the computer what to do.
- **NumPy:** calculate efficiently with arrays of numbers.
- **Pandas:** work with spreadsheet-like rows and columns.
- **Matplotlib:** turn results into readable graphs.

Example question: "How did sales change during the week?" Read the daily sales, calculate totals and averages, and plot them as a line chart.

2. Basics of programming and algorithms (00:03:53)

An **algorithm** is a finite, ordered set of instructions that solves a problem. Every program commonly has three parts:

1. **Input:** receive values (for example, hours worked and hourly pay).
2. **Process:** perform operations on those values.
3. **Output:** show or save the result.

Pseudocode is a human-readable version of an algorithm, without strict programming syntax. A **flowchart** expresses the steps using graphical shapes and arrows.

Worked example: computing employee pay

```
START
INPUT hours_worked
INPUT hourly_rate
pay = hours_worked * hourly_rate
OUTPUT pay
END
```

The matching Python program is:

```
hours_worked = 8
hourly_rate = 150
pay = hours_worked * hourly_rate
print(pay)  # 1200
```

Key idea: First clarify the problem and the steps; only then write code. A correct algorithm matters more than memorizing syntax.

Practice: Write pseudocode that reads three exam scores and prints their average.

3. Why Python? (01:11:35)

Python is popular in data science because its syntax is readable, it has a large open-source ecosystem, and notebooks allow fast experimentation. You can start with everyday calculations and gradually build analysis pipelines.

```
marks = [80, 75, 95]
average = sum(marks) / len(marks)
print(average)  # 83.33333333333333
```

The same basic Python knowledge will later support data cleaning, visualization, machine learning, and automation.

4. Install Python and Anaconda (01:33:09)

Python runs your programs. **Anaconda** is a distribution that bundles Python and data-science packages, along with environment management. It is one convenient route, not a strict requirement.

Option A - Anaconda: Install Anaconda, open Anaconda Navigator or an Anaconda Prompt, and launch Jupyter Notebook.
Option B - Standard Python + VS Code: Install Python from python.org, add the Python and Jupyter extensions in VS Code, and install the libraries below in a terminal.

```
python -m pip install numpy pandas matplotlib jupyter
```

On some systems the command is `py -m pip ...` or `python3 -m pip ...`.

Verify the environment:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
print("Ready for data science")
```

Tip: Keep each project in its own virtual environment if you work on multiple projects. Avoid mixing many unrelated packages into one shared environment.

5. Jupyter Notebook (01:37:25)

A notebook (`.ipynb`) consists of **code cells** and **Markdown cells**. A code cell executes Python; a Markdown cell stores explanations, headings, and formulas. Output appears directly below the cell.

- Run the selected cell with **Shift + Enter**.
- Variables remain available in the running kernel after a cell executes.
- Cell execution order matters: if you use a variable before defining it, you get a `NameError`.
- **Restart and Run All** is a useful test: it checks that the notebook works from top to bottom.

```
x = 10      # Run this cell first
y = x * 2  # Run this cell second
print(y)    # 20
```

6. iPython Shell (01:46:28)

IPython is an interactive Python environment. It is useful for quickly trying code, inspecting variables, and exploring libraries.

```
number = 42
print(type(number))  # <class 'int'>
```

In IPython/Jupyter, `%whos` displays information about variables currently defined. This `%` syntax is an IPython magic command, **not** ordinary Python syntax in a `.py` script.

PART II - CORE PYTHON ESSENTIALS

7. Variables, data types, and operators (01:53:33)

A **variable** is a name bound to a value. Python determines the type of a value at runtime.

Type	Example	Meaning
`int`	`age = 21`	Whole number
`float`	`price = 9.5`	Decimal number
`str`	`name = "Ada"`	Text
`bool`	`passed = True`	True or false
`NoneType`	`result = None`	No value yet

```
age = 21
height = 1.75
name = "Ada"
print(type(age))      # <class 'int'>
print(type(height))   # <class 'float'>
print(type(name))     # <class 'str'>
```

Arithmetic operators

Operator	Purpose	Example	Result
`+`	Addition	`7 + 3`	`10`
`-`	Subtraction	`7 - 3`	`4`
`*`	Multiplication	`7 * 3`	`21`
`/`	True division	`7 / 2`	`3.5`
`//`	Floor division	`7 // 2`	`3`
`%`	Remainder	`7 % 2`	`1`
`**`	Power	`2 ** 3`	`8`

Watch out: `=` assigns a value; `==` checks equality. Python follows operator precedence, so use parentheses when it improves clarity.

```
x = 5
x += 2       # Same as x = x + 2
print(x)     # 7
print((2 + 3) * 4)  # 20
```

Type conversion allows a value to be represented as a different type:

```
text = "25"
number = int(text)
print(number + 5)  # 30
```

Practice: What do `10 / 4`, `10 // 4`, and `10 % 4` return?

8. Booleans, comparisons, and logic (02:27:45)

A **Boolean expression** evaluates to `True` or `False`. It is the building block of decisions and filters.

Comparison	Description
`==` / `!=`	Equal / not equal
`<` / `>`	Less than / greater than
`<=` / `>=`	Less than or equal / greater than or equal
`and`	Both conditions must be true
`or`	At least one condition must be true
`not`	Reverses a Boolean value

```
age = 20
has_id = True
can_enter = (age >= 18) and has_id
print(can_enter)  # True
```

Truthiness: Empty strings, empty collections, zero, and `None` count as false in conditions. Non-empty strings and nonzero numbers usually count as true.

```
names = []
if not names:
    print("No names yet")
```

Important Pandas difference: For element-by-element conditions on columns, use `&`, `|`, and parentheses, not the regular Python `and` / `or` keywords.

9. Useful built-in functions (02:55:37)

These functions are part of Python and usually require no extra import:

Function	Use	Example
`print()`	Display output	`print("Hi")`
`input()`	Read text from user	`input("Name: ")`
`type()`	Check a value's type	`type(4.5)`
`len()`	Count elements	`len([1, 2, 3])`
`sum()`	Add items	`sum([2, 3, 4])`
`min()` / `max()`	Lowest / highest	`max([2, 9, 4])`
`abs()`	Absolute value	`abs(-6)`
`round()`	Round a value	`round(3.14159, 2)`
`range()`	Generate integers for loops	`range(5)`
`sorted()`	Return a sorted list	`sorted([3, 1, 2])`

```
scores = [60, 80, 70]
print(len(scores))              # 3
print(max(scores))              # 80
print(round(sum(scores) / 3, 2))  # 70.0
```

Remember: `input()` returns a `str`, even if the user types digits. Use `int()` or `float()` when a numeric value is required.

10. Control flow: conditions and loops (03:20:04)

Control flow decides which statements run and how often they run. Python uses indentation (normally four spaces) to group blocks.

`if`, `elif`, `else`

```
mark = 78
if mark >= 80:
    grade = "A+"
elif mark >= 70:
    grade = "A"
else:
    grade = "Needs improvement"
print(grade)  # A
```

`for` loop: visit each item

```
sales = [100, 150, 250]
for amount in sales:
    print(amount)
```

`while` loop: repeat while a condition holds

```
count = 1
while count <= 3:
    print(count)
    count += 1  # Prevent an infinite loop
```

- `break` stops the nearest loop immediately.
- `continue` skips the rest of the current iteration.
- `range(5)` yields `0, 1, 2, 3, 4`, not `1` through `5`.

```
total = 0
for value in [5, 10, 15]:
    total += value
print(total)  # 30
```

Practice: Print only the even numbers from 1 to 10. Then calculate their sum.

11. Functions: write reusable code (05:11:52)

A **function** groups instructions behind a meaningful name. It may accept **parameters** and return a result using `return`.

```
def calculate_average(numbers):
    if not numbers:
        return None
    return sum(numbers) / len(numbers)

result = calculate_average([10, 20, 30])
print(result)  # 20.0
```

Terminology:

- **Definition:** `def calculate_average(numbers): ...`
- **Parameter:** `numbers` in the definition.
- **Argument:** `[10, 20, 30]` passed to the function.
- **Return value:** the result sent back to the caller.

Default arguments provide a value if the caller does not supply one:

```
def add_tax(price, rate=0.05):
    return price * (1 + rate)

print(add_tax(100))       # 105.0
print(add_tax(100, 0.10)) # 110.00000000000001 (floating-point)
```

`print()` displays a result, but `return` lets other code **use** the result. Floating-point decimal values may have tiny rounding differences; use `round()` for display only when appropriate.

Scope: A variable created inside a function is normally local to that function. Avoid relying on global variables for essential calculations.

Practice: Write `is_even(n)` that returns a Boolean.

12. Modules and imports (06:41:47)

A **module** is a Python file or package containing code you can reuse. Instead of rewriting mathematical functions, import them.

```
import math
print(math.sqrt(81))  # 9.0
print(math.pi)        # 3.141592653589793
```

```
from statistics import mean
print(mean([4, 6, 8]))  # 6
```

Three import styles you will encounter:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Here `np`, `pd`, and `plt` are conventional **aliases**. They improve readability and are widely used in tutorials and real projects.

Custom module: If `my_math.py` contains `def square(x): return x * x`, another file in the same project can `import my_math` and call `my_math.square(4)`.

13. Strings: text operations (07:30:04)

A **string** stores text and supports indexing, slicing, and many helpful methods. String positions begin at zero, and strings cannot be modified in place.

```
word = "Python"
print(word[0])     # P
print(word[-1])    # n
print(word[0:3])   # Pyt; end index excluded
print(word.lower())  # python
```

```
message = "  Data Science, Python  "
clean = message.strip()
print(clean)                       # Data Science, Python
print(clean.replace("Python", "AI"))
print(clean.split(", "))          # ['Data Science', 'Python']
```

f-strings combine values and text clearly:

```
name = "Amina"
score = 88.5
print(f"{name} scored {score:.1f}%")
```

Practice: Given `" DHAKA, Bangladesh "`, remove extra outside spaces and make everything lowercase.

14. Data structures (08:23:57)

A **data structure** decides how values are stored and retrieved. Choose it based on the task.

Structure	Syntax	Ordered?	Mutable?	Duplicates?
List	`[1, 2, 2]`	Yes	Yes	Yes
Tuple	`(1, 2, 2)`	Yes	No	Yes
Set	`{1, 2}`	No meaningful position	Yes	No
Dictionary	`{"a": 1}`	Insertion order	Yes	Keys unique

Lists: flexible sequences

```
marks = [70, 80, 90]
marks.append(95)
marks[0] = 75
print(marks[1:3])  # [80, 90]
print(marks)       # [75, 80, 90, 95]
```

Tuples: fixed sequences

```
point = (10, 20)
x, y = point
print(x, y)  # 10 20
```

Sets: unique items and membership

```
cities = {"Dhaka", "Sylhet", "Dhaka"}
print(len(cities))       # 2
print("Dhaka" in cities) # True
```

Dictionaries: named fields / key-value pairs

```
student = {"name": "Amina", "score": 92}
student["grade"] = "A+"
print(student["name"])        # Amina
print(student.get("age", 0))  # 0, no KeyError
```

The big distinction: A list uses integer **positions**; a dictionary uses **keys**. In a set you test membership rather than accessing `set[0]`.

Practice: Build a dictionary for a book with `title`, `author`, and `price`. Print the price and then update it.

PART III - THE DATA SCIENCE LIBRARIES

15. NumPy: numerical arrays (09:36:10)

NumPy (Numerical Python) provides efficient multidimensional arrays. A NumPy array is useful when many values share a numeric type and you want fast mathematical operations.

```
import numpy as np

a = np.array([10, 20, 30])
print(a.shape)  # (3,)
print(a.dtype)  # e.g. int64 or int32
print(a * 2)   # [20 40 60]
```

In ordinary Python, adding two lists concatenates them. In NumPy, adding two same-shaped numeric arrays performs **element-wise addition**.

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(x + y)  # [5 7 9]
print(x * y)  # [4 10 18]
```

One-dimensional versus two-dimensional arrays

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print(matrix.shape)   # (2, 3): 2 rows, 3 columns
print(matrix.ndim)    # 2
print(matrix[0, 1])   # 2
print(matrix[:, 0])   # [1 4]: first column
```

Core array operations

Task	Example	Meaning
Create zeros	`np.zeros((2, 3))`	2x3 array of zeros
Create sequence	`np.arange(0, 10, 2)`	`[0, 2, 4, 6, 8]`
Evenly spaced values	`np.linspace(0, 1, 5)`	5 values from 0 to 1
Reshape	`np.arange(6).reshape(2, 3)`	Convert 6 values to 2x3
Mean	`np.mean(x)`	Average
Standard deviation	`np.std(x)`	Spread of values
Maximum	`np.max(x)`	Highest value

The meaning of `axis`

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.sum(axis=0))  # [5 7 9]: sum down rows, per column
print(m.sum(axis=1))  # [6 15]: sum across columns, per row
```

Broadcasting: NumPy can sometimes combine differently shaped arrays by expanding dimensions according to matching-size rules. Adding a scalar (like `a + 5`) to every array entry is the easiest example.

Common mistakes: NumPy indices start at zero; shapes must be compatible for element-wise operations; and slices can be **views**, so changing a slice may change the original array. Use `.copy()` when you need independent data.

Practice: Create a 3x2 array from 1 through 6, find its shape, column sums, and overall mean.

16. Pandas: tables and data analysis (11:04:12)

Pandas is built around two main objects:

- **Series:** a labeled, one-dimensional collection of values.
- **DataFrame:** a labeled, two-dimensional table with rows and columns.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Amina", "Rafi", "Sara"],
    "score": [85, 72, 93],
    "city": ["Dhaka", "Sylhet", "Dhaka"]
})
print(df)
```

Inspect a dataset

Command	What it tells you
`df.head()` / `df.tail()`	First / last few rows
`df.shape`	Number of rows and columns
`df.columns`	Column names
`df.info()`	Types and non-null counts
`df.describe()`	Numeric summary statistics
`df.isna().sum()`	Missing values in each column

Select rows and columns

```
print(df["score"])              # One Series
print(df[["name", "score"]])  # Two-column DataFrame
print(df.iloc[0])             # First row, by position
print(df.loc[0, "name"])     # Label-based access
```

`.iloc` selects by **integer position**. `.loc` selects by **index or column label**. They can produce different results if the DataFrame has custom index labels.

Filter and create columns

```
high_scores = df[df["score"] >= 80]
dhaka_high = df[(df["city"] == "Dhaka") & (df["score"] >= 80)]
df["passed"] = df["score"] >= 40
print(high_scores)
```

Use **parentheses** around Pandas Boolean conditions, with `&` for AND and `|` for OR.

Work with CSV files

```
df = pd.read_csv("students.csv")
print(df.head())
# After changes:
df.to_csv("students_clean.csv", index=False)
```

`read_csv()` expects a real file at the path you supply. Keep the CSV in the project directory or supply a correct full path. `index=False` prevents the DataFrame index from becoming an extra column in the exported CSV.

Missing data and simple cleaning

```
print(df.isna().sum())
df["score"] = pd.to_numeric(df["score"], errors="coerce")
df["score"] = df["score"].fillna(df["score"].median())
df = df.drop_duplicates()
```

Important: Not all missing values should be replaced with a median. Choose a strategy based on the data and meaning of the column, and document what you changed.

Group and summarize

```
summary = df.groupby("city")["score"].mean()
print(summary)
```

`groupby()` means "split the table into categories, calculate within each category, and combine the results." It is one of the most useful data-analysis tools.

Sort and merge

```
sorted_df = df.sort_values("score", ascending=False)
# Example of joining when you have another table named city_info:
# combined = df.merge(city_info, on="city", how="left")
```

Practice: Load a CSV of product sales; inspect the first five rows, remove duplicate records, and calculate average sales per product.

17. Matplotlib: visualizing results (12:01:31)

Matplotlib converts numeric results into charts. The usual alias `plt` points to its `pyplot` interface.

Line chart: trends over time

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4]
sales = [100, 150, 130, 180]
plt.plot(days, sales, marker="o", label="Daily sales")
plt.xlabel("Day")
plt.ylabel("Sales")
plt.title("Sales trend")
plt.legend()
plt.grid(alpha=0.25)
plt.show()
```

Bar chart: compare categories

```
cities = ["Dhaka", "Sylhet", "Rajshahi"]
orders = [120, 80, 65]
plt.bar(cities, orders)
plt.title("Orders by city")
plt.ylabel("Orders")
plt.show()
```

Histogram: understand distributions

```
scores = [42, 55, 61, 61, 70, 75, 88, 92]
plt.hist(scores, bins=5, edgecolor="black")
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.show()
```

Chart	Best suited for
Line	Changes over time
Bar	Comparing groups
Histogram	Distribution of a numeric variable
Scatter	Relationship between two numeric variables

Rules for good charts: label axes, use descriptive titles, choose the right chart, and avoid misleading scales. Use `plt.savefig("chart.png", dpi=150, bbox_inches="tight")` **before** `plt.show()` to save an image.

Practice: Make a bar chart from three products and their revenue, then add axis labels.

PART IV – GUIDED CAPSTONE PROJECT

18. COVID-19 trend analysis (12:09:00)

The video ends with a COVID-19 trend-analysis example that combines data loading, handling missing values, data transformations, and charting. The repository includes `covid-19_trendAnalysis.ipynb`. Some original notebook file paths point to an instructor's local machine; replace those with your own dataset path. The dataset is historical, so the notebook should be treated as a programming exercise rather than a source for current public-health statistics.

Goal

Given records containing a **date**, **country**, and **confirmed case count**, plot the trend for a chosen country.

Step 1 - Create a tiny practice CSV

Save the text below as `cases_example.csv` in your project folder. **These numbers are fictional and only illustrate the code.**

```
Date,Country,Confirmed
2020-03-01,Exampleland,10
2020-03-02,Exampleland,15
2020-03-03,Exampleland,23
2020-03-01,Otherland,4
2020-03-02,Otherland,6
2020-03-03,Otherland,11
```

Step 2 - Read, inspect, and filter

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("cases_example.csv")
print(df.head())
print(df.isna().sum())
df["Date"] = pd.to_datetime(df["Date"])
country = df[df["Country"] == "Exampleland"].copy()
country = country.sort_values("Date")
```

Step 3 - Calculate day-to-day change

```
country["NewCases"] = country["Confirmed"].diff()
print(country[["Date", "Confirmed", "NewCases"]])
```

The first `NewCases` value is `NaN` because there is no preceding date in this tiny table. In real datasets, a negative difference may reflect a data correction; it does not always indicate a physical negative count of new infections.

Step 4 - Plot the trend

```
plt.figure(figsize=(7, 4))
plt.plot(country["Date"], country["Confirmed"], marker="o")
plt.title("Illustrative cumulative cases - Exampleland")
plt.xlabel("Date")
plt.ylabel("Cumulative confirmed cases")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

If your real data contains regions or repeated country-date rows

Do not plot the raw values blindly. First determine whether rows represent distinct regions, repeated revisions, or national totals. For truly non-overlapping regional counts, grouping and summing by country and date may be appropriate; for country-level totals, summing duplicates can create double counting.

```
# Only when counts are separate, non-overlapping components:
# trend = df.groupby(["Country", "Date"], as_index=False)["Confirmed"].sum()
```

Project checklist: correct file path, proper date type, missing-value review, meaningful aggregation, chronological sorting, clear labels, and interpretation that matches the meaning of the data.

PART V - QUICK REVISION AND PRACTICE

19. Python / NumPy / Pandas / Matplotlib cheat sheet

Goal	Syntax
Print text	`print("Hello")`
Identify type	`type(x)`
Length	`len(items)`
If/else	`if condition: ... else: ...`
Loop	`for x in items:`
Function	`def f(x): return x * 2`
Create list	`a = [1, 2, 3]`
Create dictionary	`d = {"a": 1}`

Goal	Syntax
Create array	`np.array([1, 2, 3])`
Array shape	`arr.shape`
Array average	`arr.mean()`
Column sums	`arr.sum(axis=0)`
Load CSV	`pd.read_csv("file.csv")`
First five rows	`df.head()`
Inspect types	`df.info()`
Missing counts	`df.isna().sum()`
Select column	`df["column"]`
Filter rows	`df[df["value"] > 10]`
Group and sum	`df.groupby("group")["value"].sum()`
Export CSV	`df.to_csv("out.csv", index=False)`
Line chart	`plt.plot(x, y)`
Bar chart	`plt.bar(x, y)`
Chart labels	`plt.xlabel("..."); plt.ylabel("...")`

20. Six mistakes beginners should avoid

1. **Using `=` instead of `==`** in a comparison.
2. **Forgetting indentation** inside a function, loop, or `if` block.
3. **Expecting `/` to return an integer**: Python true division produces a floating-point result.
4. **Assuming `range(5)` includes five**: it starts at zero and stops before five.
5. **Using `and` with Pandas Series**: combine column conditions with `&` and parentheses.
6. **Treating missing values or dates as ordinary clean numbers/strings** without examining and converting them.

21. Mini practice test

Q1. What is the output of `print(9 // 2, 9 % 2)`?

Q2. Write an `if` statement that prints `Pass` when `mark >= 40`, otherwise `Fail`.

Q3. Create a function `square(n)` that returns the square of `n`.

Q4. How do list and tuple differ?

Q5. What is the result of `len(set([1, 2, 2, 3]))`?

Q6. What is the shape of `np.array([[1, 2, 3], [4, 5, 6]])`?

Q7. What does `axis=0` mean for summing a 2D NumPy array?

Q8. With a DataFrame called `df`, select rows where `age > 18`.

Q9. Which Pandas method summarizes numeric columns?

Q10. Which chart is usually best for showing a trend across dates?

Q11. Create a NumPy array `[2, 4, 6]` and multiply every element by 3.

Q12. Given a DataFrame with columns `department` and `salary`, calculate the mean salary for each department.

Answer key

1. `4 1`.
2. `if mark >= 40: print("Pass")` followed by `else: print("Fail")` on the next line, with correct indentation.
3. `def square(n): return n ** 2`.
4. Lists are mutable; tuples are immutable.
5. `3`.
6. `(2, 3)`.
7. Calculate a separate sum for each column (collapsing the row dimension).
8. `df[df["age"] > 18]`.
9. `df.describe()`.
10. A line chart.
11. `np.array([2, 4, 6]) * 3` gives `[6 12 18]`.
12. `df.groupby("department")["salary"].mean()`.

22. Suggested learning schedule

Session	Video chapters	Active work
1	Intro through Jupyter	Setup and a pay-calculator algorithm
2	Variables through useful functions	Ten small calculations
3	Control flow	Grade calculator and loop tasks
4	Functions and modules	Three custom functions
5	Strings and data structures	Student-record mini-program

Session	Video chapters	Active work
6	NumPy	Arrays, shapes, statistics
7	Pandas	Load, filter, clean, group CSV data
8	Matplotlib and project	Complete and explain your own chart

Best way to finish: Rebuild the final mini-project without copying the code, then explain what each line does. That is a much stronger test of understanding than merely completing the video.

References and course resources

- **Original freeCodeCamp video:** https://www.youtube.com/watch?v=LHBE6Q9XlzI
- **Official tutorial notebooks:** https://github.com/datapublishings/Course-python-data-science
- **Course overview published by freeCodeCamp:** https://www.freecodecamp.org/news/python-data-science-course-matplotlib-pandas-numpy/

Prepared as an independent learning companion. Chapter start times are provided by the course description; the exercises and illustrative datasets in this handbook are newly written for practice.